Fall Detection System

CG2028 Group 22
Nicholas Lau Hongyi A0308905B
Karthikeyan Vetrivel A0307459W

Bob



Bob falls down



Problem Statement



Both parents working



Ageing Population

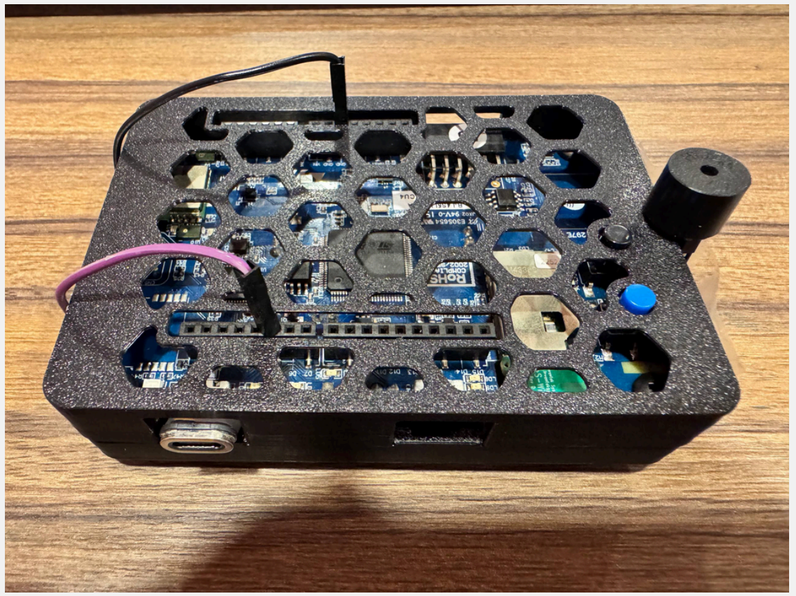


Children go school

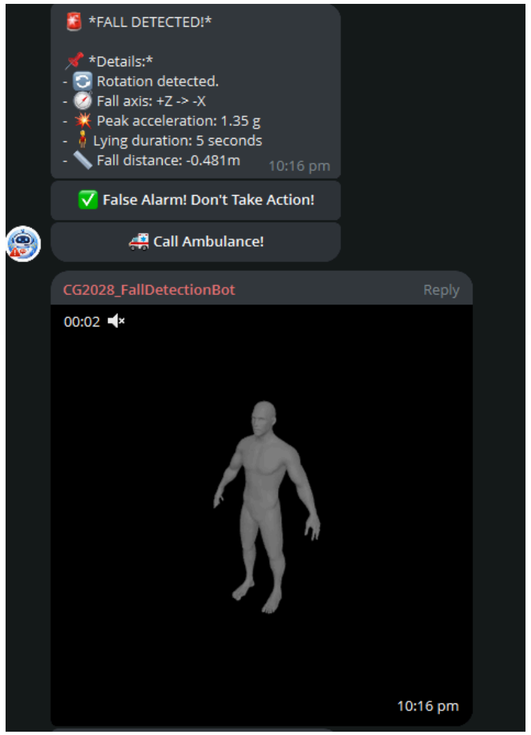


We need a better fall detection system!!

Fall Alert SysTem (FAST)



Fall Alert SysTem



System Initialisation

Calibrate her height when standing straight

STM32 board sends acc./gyro/altitude values to Server

User Falls

STM32 detects the sudden change in the axis due to fall

STM32 sends a FALL DETECTED message to Server

Server crafts a message and sends to a Telegram bot API

Telegram Bot notifies caregiver and repeatedly sends notification every 10 seconds

Caregiver calls Ambulance

Caregiver is around the elderly and knows that it is just a false alarm

User feels dizzy and unwell

User presses button to notify caregiver

STM32 sends a USER NEEDS ASSISTANCE message to Server

Server crafts a message and sends to a Telegram bot API

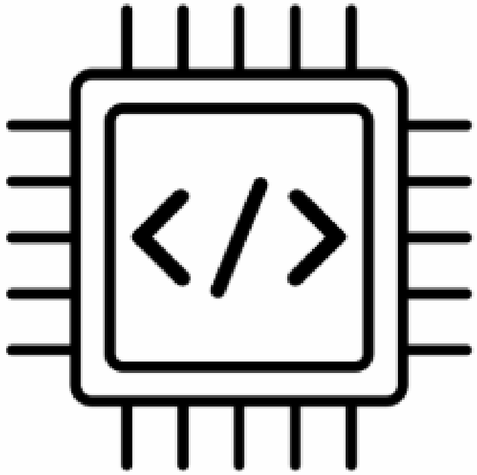
Caregiver receives the message and attends to User immediately

Fall Alert SysTem

	Actual Fall	Actual No Fall
Predicted Fall	**TP** (Correctly detected fall)	**FP** (False alarm – predicted fall but no fall happened)
Predicted No Fall	**FN** (Missed fall – system failed to detect)	**TN** (Correctly detected normal activity)

Initial Setup of System

Initial Setup



On-device side (STM32):

- **Firmware does**
  - HAL_Init()
- **Use Case:**
  - initialises the HAL driver
  - drives things like the timer, interrupt and HAL internal state

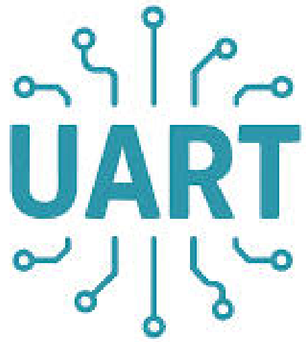
Initial Setup



On-device side (STM32):

- **Firmware does**
  - BUZZ_Init() enables the GPIOB clock and configures PB3 as output push-pull and sets buzzer off initially
- **Use Case:**
  - enable the GPIOB clock via __HAL_RCC_GPIOB_CLK_ENABLE()
  - Configure PB3 as output push-pull
  - Set buzzer off initially

Initial Setup



On-device side (STM32):

- **Firmware does**
  - UART1_Init() will enable GPIO and USART1 clocks, set PB6 and PB7 to AF for USART1 (TX/RX), Configure huart1, and calls HAL_UART_Init(&huart1)

- **Use Case:**
  - enable GPIO and USART1 clocks
  - set PB6 and PB7 to AF7 for USART1 (TX/RX)
  - configure huart1 (115200, 8N1, TX/RX)
  - calls HAL_UART_Init(&huart1)

Initial Setup

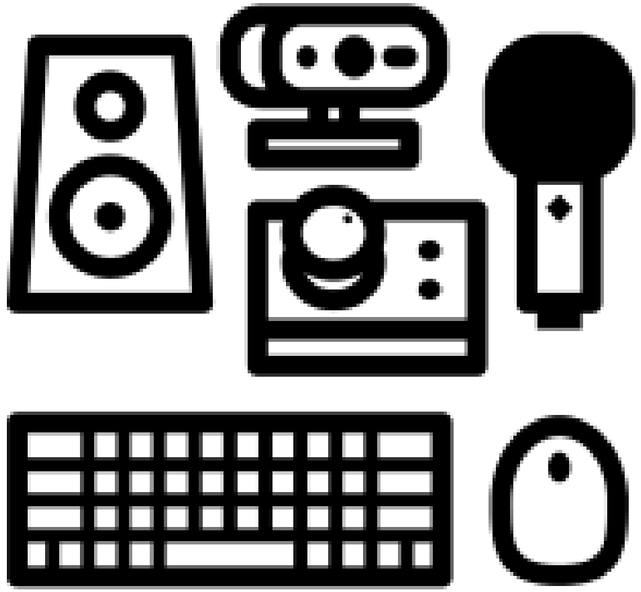


On-device side (STM32):

- **Firmware does**
  - HAL_UARTEx_ReceiveToIdle_IT(&huart1, rx_buf, sizeof(rx_buf) - 1) is called

- **Use Case:**
  - UART receives bytes into rx_buf via interrupts, filling the buffer
  - when IDLE condition happens, HAL calls HAL_UARTEx_RxEventCallback(UART_HandleTypeDef *huart, uint16_t Size)
  - where Size = how many bytes were received this "chunk"
  - it then null terminates, ignores ACK lines, and check if line starts with BEEP and contains FA or 995
  - It then rearms ReceiveToIdle again for the next line

Initial Setup



On-device side (STM32):

- **Firmware does**
  - BSP_LED_Init(LED2); # for blinking
  - BSP_ACCELERO_Init(); # accelerometer
  - BSP_GYRO_Init(); # gyroscope
  - BSP_PSENSOR_Init(); # pressure sensor
  - BSP_PB_Init(BUTTON USER, BUTTON_MODE_EXIT); # push button

STM32 is now ready to accept commands from Python

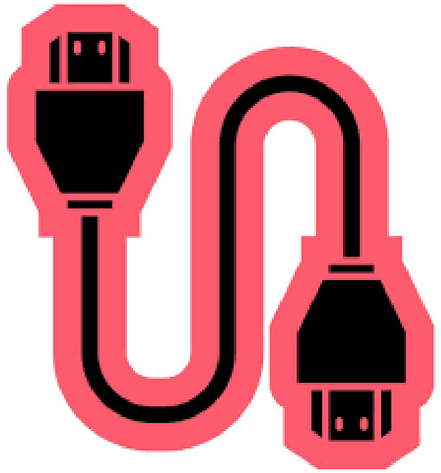
Initial Setup



On software-side (Python):

- as soon as fall_combo.py is running, configurations are set:
  - constants like BAUD, TIMEOUT_S
  - global state variables
  - Telegram token + CHAT_ID
  - Folders for frames and video output
  - Retry configuration for requests

Initial Setup



On software-side (Python):

- Port Detection
  - port is detected using auto_detect_port()
  - UART connection with baud rate 115200 is set

```
global SER
port = auto_detect_port()

# open serial
SER = serial.Serial(port, BAUD, timeout=TIMEOUT_S)
```

PC <--> STM32 Serial Link is open

Initial Setup



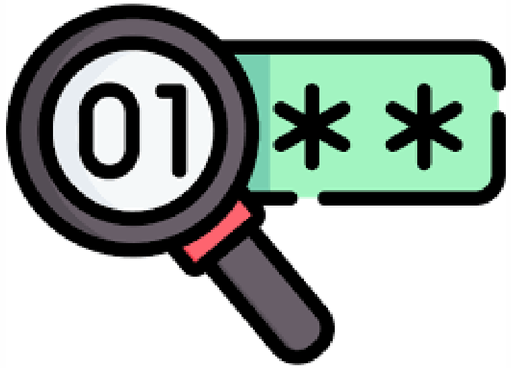
On software-side (Python):

- Telegram polling thread
  - continuously calls Telegram getUpdates() function
  - detects button presses

```
# Start telegram updates thread (daemon)
t = threading.Thread(target=telegram_updates_loop, daemon=True)
t.start()
```

System is ready to react to Telegram UI Events

Initial Setup



On software-side (Python):

- as soon as fall_combo.py is running, **function uart_loop() continuously calls ser.readline() in a loop,** and is waiting for newline-terminated UART output from STM32
- once bytes are sent over UART, it will **decode bytes sent, into Python ‘str’**
- the decoded line is then **printed to the terminal or parsed as 1 of the 3 types** (IMU CSV, Assist button text, Fall-Trigger text)

Initial Setup

On software-side (Python):

- uart_loop()
  - **Type 1:** IMU CSV - eventually write CSV clip to generate video

IMU CSV Partial Code Snippet

```
# 1) Parse IMU CSV samples for video buffer
sample = parse_imu_csv(line)
if sample is not None:
    buf.append(sample)

    if capturing_post:
        post_trigger_buffer.append(sample)

        enough = (len(post_trigger_buffer) >= post_trigger_samples_needed)
        timed_out = (
            post_capture_started_ts is not None
            and (time.time() - post_capture_started_ts) >= POST_CAPTURE_TIMEOUT_S
        )
```

Initial Setup

On software-side (Python):

- uart_loop()
  - **Type 2:** Assist text - rate limit button pressing and message spam

Assist text Code Snippet

```
# 2) Assist text trigger if its <= 1 second since last assist (prevent spam)
if ASSIST_TEXT in line:
    now = time.time()

    global assist_last_seen_ts
    if now - assist_last_seen_ts >= ASSIST_MIN_INTERVAL:
        send_message("🚨 " + ASSIST_TEXT)
        assist_last_seen_ts = now
    else:
        print("[ASSIST] Suppressed duplicate press")

    continue
```

Initial Setup

On software-side (Python):

- uart_loop()
  - **Type 3a:** Fall trigger lines - Video Trigger, which starts post-capture buffer window

Video Trigger Code Snippet

```
# 3) Trigger handling (based on the print statements)
is_video_trigger = ("!!! FALL DETECTED !!!" in line) or ("FALL DETECTED" == line)
is_data_trigger = ("FALL axis=" in line)

if not (is_video_trigger or is_data_trigger):
    print("[STM32]", line)
    continue

print("Trigger line:", line)

# VIDEO TRIGGER: start post-capture of frames
if is_video_trigger:
    capturing_post = True
    post_trigger_buffer = []
    post_capture_started_ts = time.time()
    continue
```

Initial Setup

On software-side (Python):

- uart_loop()
  - **Type 3b:** Fall trigger lines - Data Trigger, which sends telegram fall alert with buttons, upload MP4 and start reminder loop

Data Trigger Partial Code Snippet

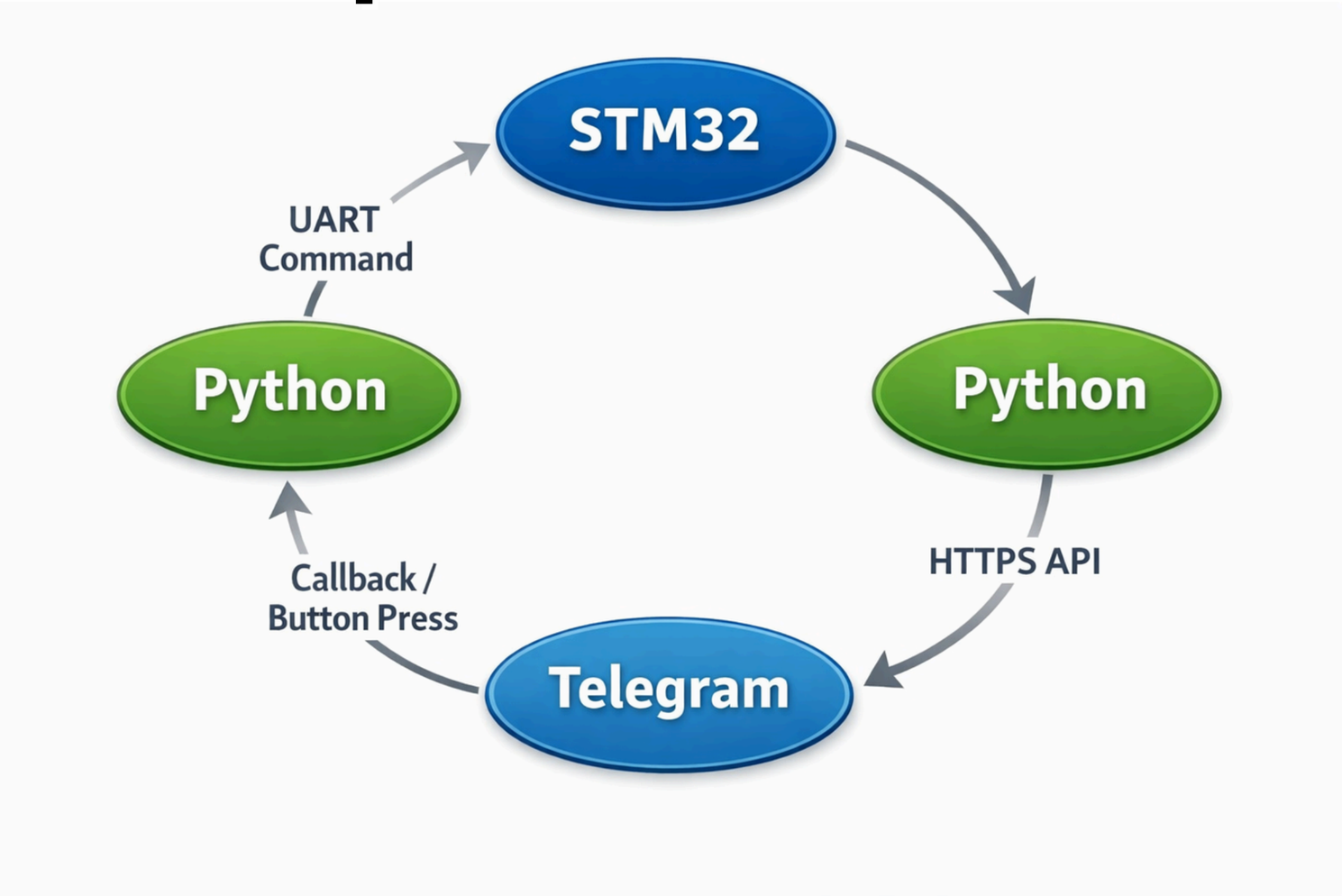
```
# DATA TRIGGER: send Telegram the message and video
if is_data_trigger:
    fall_info = parse_machine_fall_line(line)
    if not fall_info:
        continue

    now = time.time()
    if now - last_telegram_sent < TELEGRAM_COOLDOWN_S:
        continue

    alert_text = build_alert_text(
        fall_info["axis_change"],
        fall_info["peak_g"],
        fall_info["lying_s"],
        fall_info["fall_alt"] or 0
    )

    resp = send_message(alert_text, reply_markup=build_keyboard())
```

Initial Setup

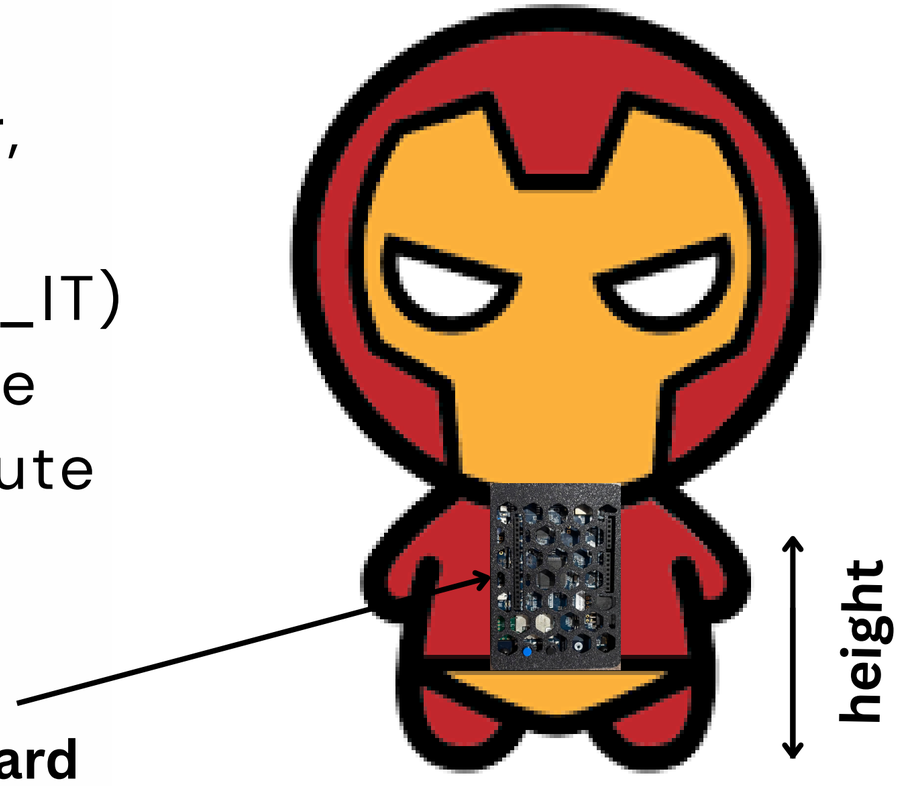


High Level Algorithm for STM32

STM32 Client High-Level Algorithm

Initialisation

1. Init UART, buzzer, LEDs, accelerometer, gyroscope, barometer
2. Arm UART RX interrupt (ReceiveToIdle_IT)
3. Wait 3 seconds for user to wear device
4. Sample barometer 100 times → compute base_pressure (calibration baseline)



STM32 Client High-Level Algorithm

Within while() loop:

Step 1: Sensor Acquisition

1. Read raw accelerometer XYZ
2. Read raw gyroscope XYZ
3. Apply moving average to accelerometer → accel_filt_asm
4. Read barometer → update EMA-filtered pressure → compute current_altitude

acc. raw readings

gyro raw readings

mov_avg to acc. readings

calculate altitude

```c
while (!stop_loop) {
    int16_t accel_data_i16[3] = { 0 };
    BSP_ACCELERO_AccGetXYZ(accel_data_i16);

    accel_buff_x[i % 4] = accel_data_i16[0];
    accel_buff_y[i % 4] = accel_data_i16[1];
    accel_buff_z[i % 4] = accel_data_i16[2];

    float gyro_data[3] = { 0.0f };
    float *ptr_gyro = gyro_data;
    BSP_GYRO_GetXYZ(ptr_gyro);

    float gyro_velocity[3] = { 0.0f };
    gyro_velocity[0] = gyro_data[0] / 1000.0f;
    gyro_velocity[1] = gyro_data[1] / 1000.0f;
    gyro_velocity[2] = gyro_data[2] / 1000.0f;

    float accel_filt_asm[3] = { 0 };
    accel_filt_asm[0] = (float)mov_avg(N, accel_buff_x) * (9.8f / 1000.0f);
    accel_filt_asm[1] = (float)mov_avg(N, accel_buff_y) * (9.8f / 1000.0f);
    accel_filt_asm[2] = (float)mov_avg(N, accel_buff_z) * (9.8f / 1000.0f);

    float raw_pressure = BSP_PSENSOR_ReadPressure();
    total_base_pressure += raw_pressure;
    baro_count++;

    if (baro_count >= SAMPLING_COUNT) {
        float avg = total_base_pressure / (float)SAMPLING_COUNT;
        filtered_pressure = PRESSURE_EMA_ALPHA * avg
                + (1.0f - PRESSURE_EMA_ALPHA) * filtered_pressure;
        current_altitude = 44330.0f
                * (1.0f - powf(filtered_pressure / base_pressure, 0.190295f));
        total_base_pressure = 0.0f;
        baro_count = 0;
    }
```

STM32 Client High-Level Algorithm

Within while() loop:

Step 2 — Motion Analysis

1. Compute acceleration magnitude |a| from filtered XYZ
2. Compute gyroscope magnitude |g| from raw gyro XYZ
3. Compute ***jerk = |Δa| / Δt***
4. Flag sudden_rot if ***|g| ≥ 220 °/s***
5. Flag sudden_jerk if ***jerk ≥ 2.5 m/s³***

compute |a|

compute |g|

|g| ≥ 220 °/s ?

compute jerk

jerk ≥ 2.5 m/s³ ?

```
float ax = accel_filt_asm[0];
float ay = accel_filt_asm[1];
float az = accel_filt_asm[2];

a_mag = sqrtf_approx(ax * ax + ay * ay + az * az);

float gx = gyro_data[0];
float gy = gyro_data[1];
float gz = gyro_data[2];
g_mag = sqrtf_approx(gx * gx + gy * gy + gz * gz);

uint8_t sudden_rot = (g_mag >= SUDDEN_GYRO_DPS_MIN);

if (!prev_a_mag_valid) {
    prev_a_mag = a_mag;
    prev_a_mag_valid = 1;
} else {
    jerk = absf(a_mag - prev_a_mag) / (SAMPLE_PERIOD_MS / 1000.0f);
    prev_a_mag = a_mag;
}

uint8_t sudden_jerk   = (jerk >= SUDDEN_JERK_MIN);
uint8_t sudden_action = (sudden_rot || sudden_jerk);
```

STM32 Client High-Level Algorithm

Within while() loop:

Step 3 — Impact Latching

1. **If** (sudden_rot OR sudden_jerk) AND ***|a| ≥ 12 m/s²***:
   - Latch impact → store peak_g, rotation_flag, jerk_flag, timestamp
2. **Else if** latch is older than 1500 ms → expire and clear latch

latch impact

if latch > 1500ms

```c
uint8_t sudden_action = (sudden_rot || sudden_jerk);

if (sudden_action && a_mag >= TRIGGER_A_MIN) {
    impact_latched = 1;
    impact_time_ms = now;
    latched_rot    = sudden_rot;
    latched_jerk   = sudden_jerk;
    latched_a_mag  = a_mag;
}

if (impact_latched && (now - impact_time_ms) > 1500) {
    impact_latched = 0;
}
```

STM32 Client High-Level Algorithm

Within while() loop:

Step 4 — Orientation Tracking

1. Check if total acceleration magnitude is near 1g (7–12.5 m/s²) → **stable**
2. Find dominant axis (X, Y, or Z) — must contribute ≥ 55% of |a| with ≥ 10% margin over 2nd axis
3. If dominant axis changed from previous:
   - Start debounce timer (150 ms candidate window)
   - If candidate ***stable for ≥ 150 ms AND cooldown ≥ 400 ms*** → axis_transition confirmed

7.5 m/s² < tot. acc. < 12.5m/s²

find dominant axis

if dominant axis changed → FALL!

```
uint8_t stable_1g = (a_mag >= G_STABLE_MIN && a_mag <= G_STABLE_MAX);

float abx = absf(ax), aby = absf(ay), abz = absf(az);
int   top1_axis = 0;
float top1 = abx;
float top2 = 0.0f;

if (aby > top1) { top2 = top1; top1 = aby; top1_axis = 1; }
else            { top2 = aby; }

if (abz > top1) { top2 = top1; top1 = abz; top1_axis = 2; }
else if (abz > top2) { top2 = abz; }

uint8_t dom_ok    = (top1 / a_mag) >= DOM_AXIS_FRAC_MIN;
uint8_t margin_ok = ((top1 - top2) / a_mag) >= AXIS_SWITCH_MARGIN_FRAC;
uint8_t axis_valid_now = stable_1g && dom_ok && margin_ok;

if (axis_valid_now) {
    float raw_vals[] = { ax, ay, az };
    char current_sign = (raw_vals[top1_axis] >= 0) ? '+' : '-';
    if (!prev_axis_valid) {
        prev_axis       = top1_axis;
        prev_sign       = current_sign;
        prev_axis_valid = 1;
        axis_candidate  = top1_axis;
        axis_candidate_ms = now;
    } else {
        if (top1_axis != prev_axis) {
            if (axis_candidate != top1_axis) {
                axis_candidate    = top1_axis;
                axis_candidate_ms = now;
            }
            process_axis_transition(now, top1_axis, current_sign,
                    base_pressure, filtered_pressure);
        } else {
            axis_candidate    = top1_axis;
            axis_candidate_ms = now;
        }
    }
}
```

STM32 Client High-Level Algorithm

Within while() loop:

Step 5 — Fall Decision

1. If ***axis_transition confirmed AND impact_latched***:
    - FALL DETECTED → Transmit over UART
    - Set state → FALL_SUSPECT, start 3-second fast-blink timer
2. Else → update prev_axis, continue

Step 6 — Post-Fall Window (3 seconds)

1. LED blinks fast (100 ms toggle)
2. Awaiting caregiver response via Telegram (handled by Python server)
3. After 3 seconds → reset state to FALL_NORMAL, continue detection

```
Trigger line: !!! FALL DETECTED !!!
Trigger line: FALL axis=+Z -> -X peak_g=1.28 lying_s=5 fall_alt=-1.268
[STM32] TRIGGER  : ROT + JERK
[STM32] Direction: +Z -> -X | Acc: 12.58 g
Video saved -> C:\Users\vetri\STM32CubeIDE\CG2028workspace\serverSide_v2\frames_out\fa
ll_20260222_222039\out.mp4
Frames saved -> C:\Users\vetri\STM32CubeIDE\CG2028workspace\serverSide_v2\frames_out\f
all_20260222_222039
Rendered video -> C:\Users\vetri\STM32CubeIDE\CG2028workspace\serverSide_v2\videos_out
\fall_20260222_222039.mp4
[STM32] Resuming detection
```

STM32 Client High-Level Algorithm

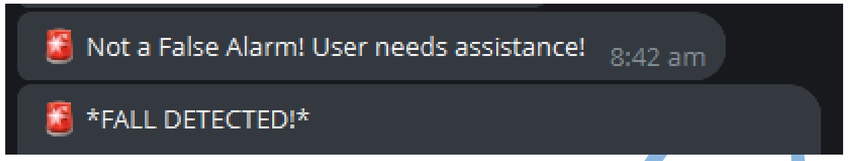
Parallel Interrupts:

UART RX Interrupt

1. Line received → check prefix:
   a. ***ACK*** → ignore (Python handshake)
   b. ***PAT_FA*** → Arm buzzer: · · · (3 short beeps — false alarm dismissed)
   c. ***PAT_995*** → Arm buzzer: — · — · — (long-short alternating — emergency escalation)

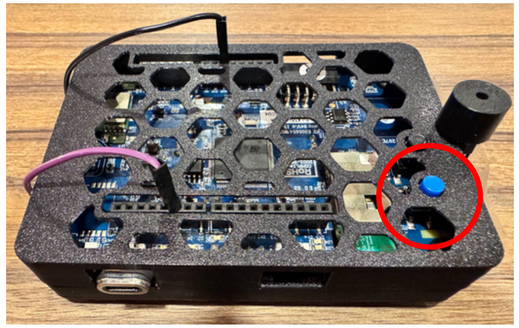
Button Interrupt

1. User presses physical button → transmit ***"Not a False Alarm! User needs assistance!"***
2. Arm buzzer: · · · — — — · · · (Morse SOS — universal distress signal)



These interrupts will **pre-empt** the acc./gyro. transmission!

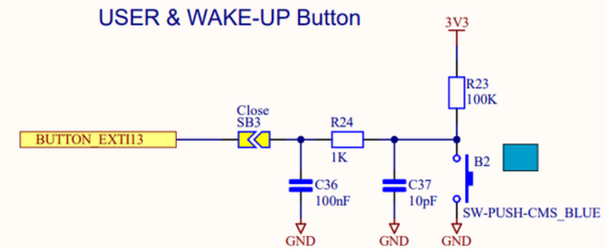
Peripheral Setup





Button Press Logic:

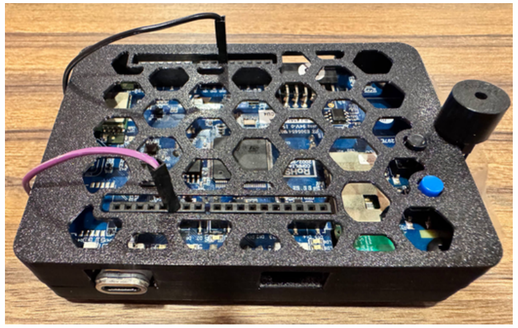
- BSP_PB_Init(BUTTON_USER, BUTTON_MODE_EXTI);
- Configures the USER button pin to trigger an external interrupt when pressed

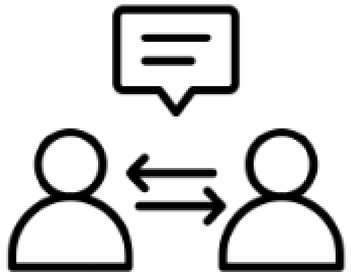


Process:

- pressing the blue button causes a voltage change on the USER button GPIO line [Active Low]
- EXTI logic detects an rising edge
- CPU jumps to the EXTI interrupt handler where message is sent via UART

UART RX





```
void HAL_UARTEx_RxEventCallback(UART_HandleTypeDef *huart, uint16_t Size);
```

Configures USART1 to trigger an interrupt when a full transmission is received

Process:

1. Incoming bytes arrive at **PB7 RX pin** → shifted to the **USART1 data register**
2. HAL accumulates bytes into ***rx_buf*** automatically via interrupt
3. When the UART line goes idle (no new bytes for one frame period), the USART1 idle detection logic fires ***[HAL_UARTEx_ReceiveToIdle_IT]***
4. CPU jumps to USART1_IRQHandler
5. HAL routes to ***HAL_UARTEx_RxEventCallback*** where the complete line is parsed
6. ***buzzer_req*** variable is set
7. ***ReceiveToIdle_IT*** is re-armed at the end of the callback to listen for the next line

Extensions

Extensions

1. **Telegram Bot** - For users to get notified when the user has fallen

2. **Telegram Group Chat** - To add family members so everyone is notified

3. **3D rendering** - Generate a 3D visualisation of how the person falls

4. **Use of Barometer** - Determine fall distance

Extensions

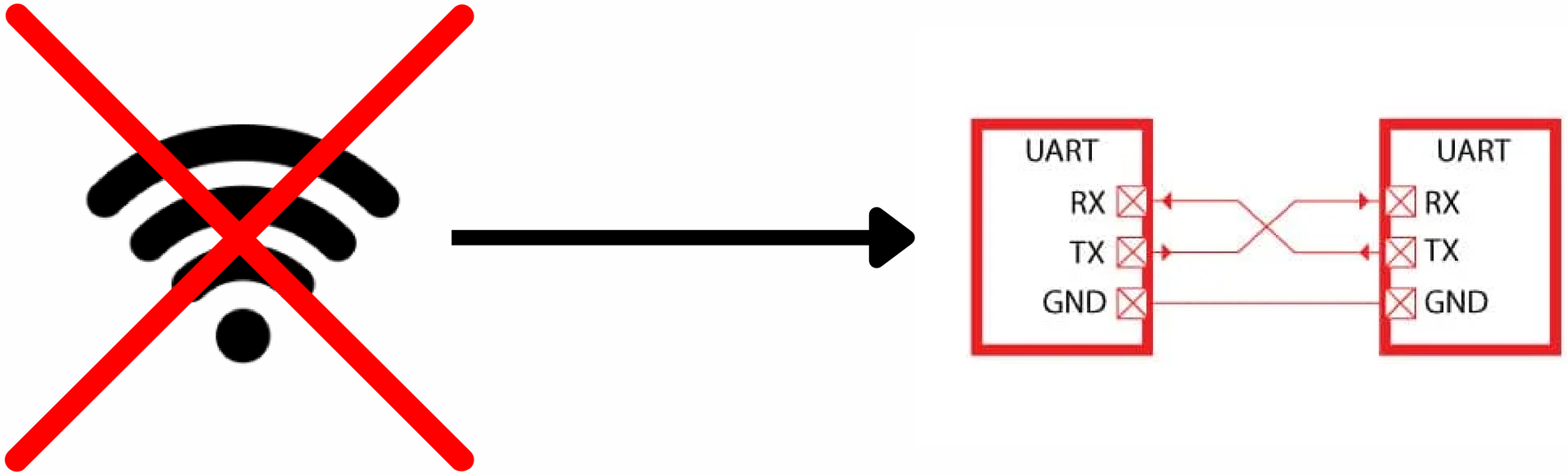
4. **User Button** - Notify family member when elderly feels unwell, and needs attention

5. **Buzzer** - Buzzes to notify the faller that either:
    - **help is on the way** or
    - that the family member has dismissed it as a **false alarm**

6. **Additional ASM file** created to execute square root (sqrtf_approx)

Problems Encountered

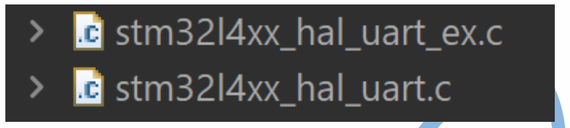
Figuring out the System Architecture



Figuring out the RX in STM32

- Created a separate code to debug UART handshake (comms_check.py)
- Had to read the given stm32 HAL files to understand
  - Two existing interrupt functions
    - HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
    - HAL_UARTEx_RxEventCallback(UART_HandleTypeDef *huart, uint16_t Size)
- Although buzzer_req was called by multiple functions it was not declared **volatile**

```
volatile pattern_t buzzer_req = PAT_NONE;
```



Figuring out Video Rendering

- Open Source wasn’t the best choice for our application
- Came up with our own solution
- Multiple Frames = Videos
- Plot using **matplotlib** → **pyrender**



Appendix: Roles and Contributions

Member	Contribution to Project
Nicholas Lau Hongyi	• Create initial fall detection system • Create Telegram bot and code for bi-directional communication between STM32 and Telegram • Optimize video generation and created code to send video via Telegram • Git Revision Control
Karthikeyan Vetrivel	• Optimise fall detection system and messages sent • Created 3D orientation visualisation • Create video of fall using csv file and multiple frames collected using a python script • Create Buzzer System and coded button/UART interrupt • Create script to test for UART communication • Git Revision Control

Thank you very much!